STORY 11.5 - HIT A SIXER GAMIFICATION SYSTEM

Complete Technical Specification & Implementation Guide

Sprint: 12
Story Points: 13
Priority: High
Type: Feature - Gamification
Dependencies: User authentication, Subscription system (Story 11.1), Payment integration

---

EPIC CONTEXT

Build a cricket-themed daily gamification system ("Hit a Sixer") that:

- Is available ONLY to paid subscribers (₹99/month platform fee)
- Awards wallet credits based on probability (8% FOUR=₹50, 4% SIXER=₹100, 88% OUT=₹10)
- Allows one play per user per day (resets midnight IST)
- Must be played before 6:00 PM IST
- Credits expire 30 days after earning
- Credits auto-apply at session booking checkout

USER STORY

As a MANAS360 subscriber
I want to play a daily cricket game to win wallet credits
So that I can reduce my therapy session costs and feel rewarded for being a subscriber

---

BUSINESS VALUE

Revenue Impact

- Subscription revenue: ₹99K/month (1,000 subscribers)
- Game cost: ₹5.04L/month (1,000 plays daily)
- Session revenue: ₹73.35L/month
- Net profit: ₹70.98L/month
- **ROI: 2,112%**

User Retention

- Expected retention: 95%+
- Daily engagement: 80%+ of subscribers
- LTV increase: ₹2,376 (24-month average)

Behavioral Goals

- Create daily habit (play every day)
- Drive subscription conversions (game as premium feature)
- Increase booking frequency (use credits before expiry)
- Reduce perceived session cost (₹999 → ₹549 average)

ACCEPTANCE CRITERIA

Functional Requirements

✅ **AC1: Subscription Gate**

- GIVEN a user is NOT subscribed
- WHEN they attempt to access the cricket game
- THEN they see "Subscribe to Play" screen with upgrade prompt
- AND game is locked/disabled

✅ **AC2: Daily Play Limit**

- GIVEN a subscribed user
- AND they have already played today
- WHEN they attempt to play again
- THEN they see "Already played today. Come back tomorrow!" message

- AND cannot play until midnight IST

✅ **AC3: Time Window Enforcement**

- GIVEN current time is after 6:00 PM IST
- WHEN user attempts to play
- THEN they see "Game closed for today. Play tomorrow before 6 PM!"
- AND game is disabled

✅ **AC4: Probability Distribution**

- GIVEN 1,000 plays
- WHEN results are aggregated
- THEN FOUR appears 75-85 times (7.5-8.5%)
- AND SIXER appears 35-45 times (3.5-4.5%)
- AND OUT appears 875-890 times (87.5-89%)

✅ **AC5: Wallet Credit Award**

- GIVEN user wins FOUR
- WHEN game result is processed
- THEN ₹50 is added to their wallet INSTANTLY
- AND transaction is logged
- AND credit expires in exactly 30 days

✅ **AC6: Credit Expiry**

- GIVEN a wallet credit is 30 days old
- WHEN expiry cron job runs
- THEN credit is marked as expired
- AND removed from available balance
- AND user is notified (if >₹50 expiring)

✅ **AC7: Checkout Integration**

- GIVEN user has ₹100 in wallet
- WHEN they book a ₹999 session
- THEN wallet credits auto-apply
- AND final amount is ₹899
- AND wallet balance reduces to ₹0

✅ **AC8: Transaction Tracking**

- GIVEN any wallet activity
- WHEN credit is earned, used, or expired
- THEN transaction is logged with timestamp
- AND user can view complete history
- AND balance is updated atomically

TECHNICAL ARCHITECTURE

Tech Stack

- **Backend:** Python 3.11+ with FastAPI OR Node.js 18+ with Express
- **Database:** PostgreSQL 15+
- **Cache:** Redis 7+ (for rate limiting, daily play tracking)
- **Queue:** Celery (Python) or Bull (Node.js) for background jobs
- **Randomization:** Python `secrets` module or Node.js `crypto` (cryptographically secure)
- **Timezone:** pytz or moment-timezone for IST handling

System Components

```
┌──────────────
```

```
│              ┌──────────────────────────┐              │
│              │ Game Engine Service │              │
│              │ - Eligibility check │              │
│              │ - Probability calc  │              │
│              │ - Result generation │              │
│              └──────────────────────────┘              │
│                     │                  │
│              ┌──────────────────┴──────────────────┐          │
│              ▼                  ▼          │
│       ┌──────────────────────┐   ┌──────────────────────┐   │
│       │ Wallet Service  │   │ Redis Cache    │   │
│       │ - Add credits   │   │ - Daily plays  │   │
│       │ - Track balance │   │ - Rate limits  │   │
│       │ - Handle expiry │   └──────────────────────┘   │
│       └──────────────────────┘                    │
│              │                          │
│              ▼                          │
│       ┌──────────────────────┐                    │
│       │  PostgreSQL    │                    │
│       │  - game_plays  │                    │
│       │  - wallet      │                    │
```

```
│        │  - transactions │                        │
│        └──────────────────────┘                    │
│                                        │
│  ┌──────────────────────────────────────────────────────┐  │
│  │          Background Jobs (Celery)              │  │
│  ├──────────────────────────────────────────────────────┤  │
│  │  - Expire credits (daily midnight)             │  │
│  │  - Send expiry warnings (7 days before)        │  │
│  │  - Generate daily stats                        │  │
│  │  - Send morning reminders (10 AM)              │  │
│  └──────────────────────────────────────────────────────┘  │
└──────────────────────────────────────────────────────────────────────┘
```

DATABASE SCHEMA

Table 1: `daily_game_plays`

Purpose: Track each user's daily game play and result

```
CREATE TABLE daily_game_plays (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    game_date DATE NOT NULL,
    played_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
```

```
-- Subscription validation
subscription_id INTEGER REFERENCES subscriptions(id),
subscription_active BOOLEAN NOT NULL DEFAULT FALSE,

-- Game outcome
outcome VARCHAR(10) NOT NULL CHECK (outcome IN ('sixer', 'four', 'out')),
credit_amount INTEGER NOT NULL, -- 100, 50, or 10 (in rupees)

-- Wallet reference
wallet_transaction_id INTEGER REFERENCES wallet_transactions(id),

-- Metadata
ip_address INET,
user_agent TEXT,
device_fingerprint VARCHAR(255),

created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

-- Constraints
CONSTRAINT unique_user_game_date UNIQUE (user_id, game_date),
```

```
    CONSTRAINT valid_credit_amount CHECK (credit_amount IN (10, 50, 100)),


    -- Indexes

    INDEX idx_user_game_date (user_id, game_date DESC),

    INDEX idx_game_date (game_date),

    INDEX idx_outcome (outcome, game_date),

    INDEX idx_subscription (subscription_id, subscription_active)

);

COMMENT ON TABLE daily_game_plays IS 'Tracks each users daily cricket game play and outcome';

COMMENT ON COLUMN daily_game_plays.outcome IS 'Game result: sixer (4%), four (8%), out (88%)';

COMMENT ON COLUMN daily_game_plays.credit_amount IS 'Wallet credit awarded: 100 for sixer, 50 for four, 10 for out';
```

Table 2: `user_wallet`

Purpose: Store user's current wallet balance and aggregates

```
CREATE TABLE user_wallet (

    id SERIAL PRIMARY KEY,

    user_id INTEGER UNIQUE NOT NULL REFERENCES users(id) ON DELETE CASCADE,


    -- Balances (in rupees)

    total_balance INTEGER NOT NULL DEFAULT 0 CHECK (total_balance >= 0),
```

```
    game_credits INTEGER NOT NULL DEFAULT 0 CHECK (game_credits >= 0),

    referral_credits INTEGER NOT NULL DEFAULT 0 CHECK (referral_credits >= 0),

    promo_credits INTEGER NOT NULL DEFAULT 0 CHECK (promo_credits >= 0),


    -- Lifetime tracking

    lifetime_earned INTEGER NOT NULL DEFAULT 0,

    lifetime_spent INTEGER NOT NULL DEFAULT 0,

    lifetime_expired INTEGER NOT NULL DEFAULT 0,


    -- Timestamps

    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    last_transaction_at TIMESTAMP WITH TIME ZONE,


    -- Indexes

    INDEX idx_user_balance (user_id, total_balance),

    INDEX idx_updated_at (updated_at DESC)

);

COMMENT ON TABLE user_wallet IS 'User wallet balances and lifetime statistics';

COMMENT ON COLUMN user_wallet.total_balance IS 'Current available balance across all
credit types';
```

Table 3: `wallet_transactions`

Purpose: Complete audit trail of all wallet activity

```
CREATE TABLE wallet_transactions (

    id SERIAL PRIMARY KEY,

    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,

    wallet_id INTEGER NOT NULL REFERENCES user_wallet(id) ON DELETE CASCADE,



    -- Transaction type

    transaction_type VARCHAR(30) NOT NULL CHECK (

        transaction_type IN (

            'credit_earned_game_four',

            'credit_earned_game_sixer',

            'credit_earned_game_out',

            'credit_earned_referral',

            'credit_earned_promo',

            'credit_used_booking',

            'credit_expired',

            'credit_refunded'

        )

    ),
```

```
-- Amount (positive for earned, negative for used/expired)
amount INTEGER NOT NULL,

-- Source reference
source VARCHAR(30), -- 'game', 'referral', 'promo', 'booking'
source_id INTEGER, -- game_play_id, referral_id, booking_id, etc.

-- Balance snapshot
balance_before INTEGER NOT NULL,
balance_after INTEGER NOT NULL,

-- Expiry tracking (for earned credits)
expires_at TIMESTAMP WITH TIME ZONE,
expired BOOLEAN DEFAULT FALSE,

-- Description
description TEXT,

-- Metadata
metadata JSONB,

created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
```

```
    -- Indexes

    INDEX idx_user_transactions (user_id, created_at DESC),

    INDEX idx_transaction_type (transaction_type, created_at DESC),

    INDEX idx_source (source, source_id),

    INDEX idx_expiry (expires_at, expired) WHERE expires_at IS NOT NULL,

    INDEX idx_created_at (created_at DESC)

);

COMMENT ON TABLE wallet_transactions IS 'Complete audit trail of all wallet credit
movements';

COMMENT ON COLUMN wallet_transactions.amount IS 'Positive for credits earned, negative
for credits used or expired';
```

Table 4: `wallet_credits`

Purpose: Individual credit line items with expiry tracking

```
CREATE TABLE wallet_credits (

    id SERIAL PRIMARY KEY,

    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,



    -- Credit details

    amount INTEGER NOT NULL CHECK (amount > 0),

    remaining_balance INTEGER NOT NULL CHECK (remaining_balance >= 0),
```

```
-- Source
source VARCHAR(30) NOT NULL, -- 'game_four', 'game_sixer', 'game_out'
source_id INTEGER, -- game_play_id
transaction_id INTEGER REFERENCES wallet_transactions(id),

-- Expiry
earned_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
expires_at TIMESTAMP WITH TIME ZONE NOT NULL,
expired BOOLEAN DEFAULT FALSE,

-- Usage
fully_used BOOLEAN DEFAULT FALSE,

-- Timestamps
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

-- Constraints
CONSTRAINT valid_remaining CHECK (remaining_balance <= amount),

-- Indexes
```

```
    INDEX idx_user_active (user_id, expired, fully_used) WHERE NOT expired AND NOT fully_used,

    INDEX idx_expiry_pending (expires_at) WHERE NOT expired,

    INDEX idx_source (source, source_id)

);

COMMENT ON TABLE wallet_credits IS 'Individual credit line items with expiry and partial usage tracking';

COMMENT ON COLUMN wallet_credits.remaining_balance IS 'Current balance of this credit line (supports partial usage)';
```

Table 5: `game_stats_daily`

Purpose: Aggregated daily statistics for monitoring

```
CREATE TABLE game_stats_daily (

    id SERIAL PRIMARY KEY,

    stat_date DATE UNIQUE NOT NULL,



    -- Play counts

    total_plays INTEGER DEFAULT 0,

    unique_players INTEGER DEFAULT 0,



    -- Outcome distribution

    total_sixers INTEGER DEFAULT 0,

    total_fours INTEGER DEFAULT 0,
```

```
    total_outs INTEGER DEFAULT 0,

    -- Percentages (for validation)
    sixer_percentage DECIMAL(5,2),
    four_percentage DECIMAL(5,2),
    out_percentage DECIMAL(5,2),

    -- Financial tracking
    total_credits_issued INTEGER DEFAULT 0, -- in rupees
    total_credits_redeemed INTEGER DEFAULT 0,
    total_credits_expired INTEGER DEFAULT 0,

    -- Session impact
    sessions_booked_with_credits INTEGER DEFAULT 0,
    revenue_generated INTEGER DEFAULT 0,

    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    INDEX idx_stat_date (stat_date DESC)
);
```

```
COMMENT ON TABLE game_stats_daily IS 'Daily aggregated statistics for monitoring and analytics';
```

Table 6: `booking_wallet_usage`

Purpose: Track wallet credit usage in bookings

```
CREATE TABLE booking_wallet_usage (

    id SERIAL PRIMARY KEY,

    booking_id INTEGER UNIQUE NOT NULL REFERENCES bookings(id) ON DELETE CASCADE,

    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,



    -- Pricing breakdown

    original_amount INTEGER NOT NULL,

    wallet_credits_used INTEGER NOT NULL,

    final_amount INTEGER NOT NULL,



    -- Credit sources used (array of wallet_credit IDs)

    credit_ids INTEGER[] NOT NULL,



    -- Transaction reference

    wallet_transaction_id INTEGER REFERENCES wallet_transactions(id),



    applied_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
```

```
    -- Indexes

    INDEX idx_booking (booking_id),

    INDEX idx_user (user_id, applied_at DESC)

);

COMMENT ON TABLE booking_wallet_usage IS 'Tracks which credits were used for which bookings';
```

BACKEND CODE (PYTHON/FASTAPI)

File: `models/game.py`

```
from enum import Enum

from datetime import datetime, timedelta

from typing import Optional

from pydantic import BaseModel, Field, validator

import pytz

class GameOutcome(str, Enum):

    """Game outcome types"""

    SIXER = "sixer"

    FOUR = "four"

    OUT = "out"

class GameOutcomeConfig:
```

```
"""Probability configuration"""
OUTCOMES = {
    GameOutcome.SIXER: {
        'probability': 0.04,  # 4%
        'credit_amount': 100,
        'display_name': 'SIXER',
        'emoji': '🚀'
    },
    GameOutcome.FOUR: {
        'probability': 0.08,  # 8%
        'credit_amount': 50,
        'display_name': 'FOUR',
        'emoji': '🎯'
    },
    GameOutcome.OUT: {
        'probability': 0.88,  # 88%
        'credit_amount': 10,
        'display_name': 'OUT',
        'emoji': '💰'
    }
}
```

```
    # Time constraints
    IST = pytz.timezone('Asia/Kolkata')
    GAME_CLOSE_HOUR = 18  # 6 PM
    GAME_CLOSE_MINUTE = 0
class GamePlayRequest(BaseModel):
    """Request to play the game"""
    user_id: int = Field(..., gt=0)
class GamePlayResponse(BaseModel):
    """Game play result"""
    success: bool
    outcome: Optional[GameOutcome] = None
    prize: Optional[dict] = None
    wallet: Optional[dict] = None
    message: str
    monthly_stats: Optional[dict] = None
class WalletBalance(BaseModel):
    """Wallet balance response"""
    user_id: int
    total_balance: int
    game_credits: int
    referral_credits: int
```

```
    promo_credits: int

    lifetime_earned: int

    lifetime_spent: int

    lifetime_expired: int

class WalletTransaction(BaseModel):

    """Wallet transaction"""

    id: int

    transaction_type: str

    amount: int

    balance_before: int

    balance_after: int

    description: str

    created_at: datetime

    expires_at: Optional[datetime] = None
```

File: `services/game_engine.py`

```
import secrets

from datetime import datetime, date, timedelta

from typing import Tuple, Optional

import pytz

from sqlalchemy.orm import Session

from models.game import GameOutcome, GameOutcomeConfig
```

```
from models.database import DailyGamePlay, UserWallet, WalletTransaction, WalletCredit

from services.subscription_service import SubscriptionService

class GameEngine:

    """Core game logic and probability engine"""


    def __init__(self, db: Session):

        self.db = db

        self.config = GameOutcomeConfig

        self.subscription_service = SubscriptionService(db)


    def check_eligibility(self, user_id: int) -> Tuple[bool, Optional[str], Optional[dict]]:

        """

        Check if user is eligible to play today


        Returns:

            (eligible: bool, error_message: str, data: dict)

        """

        # 1. Check subscription

        is_subscribed, subscription = self.subscription_service.is_active_subscriber(user_id)

        if not is_subscribed:

            return False, "SUBSCRIPTION_REQUIRED", {
```

```
        "upgrade_prompt": {
            "plan": "Premium",
            "price": 99,
            "benefits": [
                "Play daily cricket game",
                "Win ₹10-₹100 credits daily",
                "Average ₹450/month winnings",
                "Reduce session costs by 45%"
            ]
        }
    }

# 2. Check if already played today
today = self._get_ist_date()
existing_play = self.db.query(DailyGamePlay).filter(
    DailyGamePlay.user_id == user_id,
    DailyGamePlay.game_date == today
).first()

if existing_play:
    return False, "ALREADY_PLAYED_TODAY", {
        "today_result": {
```

```
            "outcome": existing_play.outcome,
            "amount_won": existing_play.credit_amount
        },
        "next_play_at": self._get_next_play_time()
    }

# 3. Check time window (before 6 PM IST)
ist_now = self._get_ist_now()
close_time = ist_now.replace(
    hour=self.config.GAME_CLOSE_HOUR,
    minute=self.config.GAME_CLOSE_MINUTE,
    second=0,
    microsecond=0
)

if ist_now >= close_time:
    return False, "GAME_CLOSED", {
        "message": "Game closed for today. Play tomorrow before 6 PM!",
        "next_play_at": self._get_next_play_time()
    }
```

```
    # All checks passed
    time_remaining = close_time - ist_now
    return True, None, {
        "time_remaining_seconds": int(time_remaining.total_seconds()),
        "closes_at": close_time.isoformat()
    }


def generate_outcome(self) -> Tuple[GameOutcome, int]:
    """
    Generate game outcome using cryptographically secure randomization


    Returns:
        (outcome: GameOutcome, credit_amount: int)
    """
    # Use secrets module for cryptographically secure random
    rand = secrets.SystemRandom()


    # Generate random float [0.0, 1.0)
    random_value = rand.random()


    # Determine outcome based on probabilities
    cumulative = 0.0
```

```
    for outcome, config in self.config.OUTCOMES.items():
        cumulative += config['probability']
        if random_value < cumulative:
            return outcome, config['credit_amount']

    # Fallback (should never reach here)
    return GameOutcome.OUT, self.config.OUTCOMES[GameOutcome.OUT]['credit_amount']

def play_game(self, user_id: int, ip_address: str = None,
              user_agent: str = None) -> dict:
    """
    Execute game play for user

    Returns:
        Game result dictionary
    """
    # Check eligibility
    eligible, error, data = self.check_eligibility(user_id)
    if not eligible:
        return {
            "success": False,
```

```
        "error": error,
        "data": data
    }

# Get subscription for tracking
_, subscription = self.subscription_service.is_active_subscriber(user_id)

# Generate outcome
outcome, credit_amount = self.generate_outcome()

# Record game play
game_play = DailyGamePlay(
    user_id=user_id,
    game_date=self._get_ist_date(),
    played_at=self._get_ist_now(),
    subscription_id=subscription.id if subscription else None,
    subscription_active=True,
    outcome=outcome.value,
    credit_amount=credit_amount,
    ip_address=ip_address,
    user_agent=user_agent
)
```

```
self.db.add(game_play)
self.db.flush()  # Get game_play.id

# Add credits to wallet
wallet_data = self._add_wallet_credit(
    user_id=user_id,
    amount=credit_amount,
    source=f"game_{outcome.value}",
    source_id=game_play.id
)

# Update game play with transaction reference
game_play.wallet_transaction_id = wallet_data['transaction_id']

self.db.commit()

# Get monthly stats
monthly_stats = self._get_monthly_stats(user_id)

# Build response
outcome_config = self.config.OUTCOMES[outcome]
```

```
        return {
            "success": True,
            "outcome": outcome.value,
            "prize": {
                "amount": credit_amount,
                "description": f"₹{credit_amount} added to wallet",
                "display_name": outcome_config['display_name'],
                "emoji": outcome_config['emoji'],
                "expires_in_days": 30,
                "expires_at": wallet_data['expires_at'].isoformat()
            },
            "wallet": {
                "previous_balance": wallet_data['balance_before'],
                "credit_added": credit_amount,
                "new_balance": wallet_data['balance_after']
            },
            "monthly_stats": monthly_stats,
            "message": f"{outcome_config['emoji']} {outcome_config['display_name']}!
₹{credit_amount} added to your wallet!"
        }


    def _add_wallet_credit(self, user_id: int, amount: int,
```

```
                    source: str, source_id: int) -> dict:
    """Add credit to user's wallet"""
    from services.wallet_service import WalletService
    wallet_service = WalletService(self.db)

    return wallet_service.add_credit(
        user_id=user_id,
        amount=amount,
        source=source,
        source_id=source_id,
        expires_in_days=30
    )

def _get_monthly_stats(self, user_id: int) -> dict:
    """Get user's monthly game statistics"""
    today = self._get_ist_date()
    month_start = today.replace(day=1)

    plays = self.db.query(DailyGamePlay).filter(
        DailyGamePlay.user_id == user_id,
        DailyGamePlay.game_date >= month_start
    ).all()
```

```
    total_won = sum(p.credit_amount for p in plays)

    # Get wallet to calculate spent
    wallet = self.db.query(UserWallet).filter(
        UserWallet.user_id == user_id
    ).first()

    subscription_fee = 99
    net_profit = total_won - subscription_fee if total_won > subscription_fee else 0
    roi_percentage = int((net_profit / subscription_fee) * 100) if subscription_fee > 0 else 0

    return {
        "games_played": len(plays),
        "total_won": total_won,
        "subscription_fee": subscription_fee,
        "net_profit": net_profit,
        "roi_percentage": roi_percentage
    }

def _get_ist_now(self) -> datetime:
```

```
        """Get current time in IST"""
        return datetime.now(self.config.IST)

    def _get_ist_date(self) -> date:
        """Get current date in IST"""
        return self._get_ist_now().date()

    def _get_next_play_time(self) -> str:
        """Get next available play time (tomorrow midnight IST)"""
        ist_now = self._get_ist_now()
        tomorrow = ist_now + timedelta(days=1)
        next_play = tomorrow.replace(hour=0, minute=0, second=0, microsecond=0)
        return next_play.isoformat()
```

File: `services/wallet_service.py`

```
from datetime import datetime, timedelta
from typing import List, Optional
import pytz
from sqlalchemy import and_
from sqlalchemy.orm import Session
from models.database import UserWallet, WalletTransaction, WalletCredit

class WalletService:
```

```
"""Wallet management service"""

def __init__(self, db: Session):
    self.db = db
    self.IST = pytz.timezone('Asia/Kolkata')

def get_or_create_wallet(self, user_id: int) -> UserWallet:
    """Get or create user wallet"""
    wallet = self.db.query(UserWallet).filter(
        UserWallet.user_id == user_id
    ).first()

    if not wallet:
        wallet = UserWallet(user_id=user_id)
        self.db.add(wallet)
        self.db.commit()

    return wallet

def add_credit(self, user_id: int, amount: int, source: str,
               source_id: int, expires_in_days: int = 30) -> dict:
```

```
"""
Add credit to user's wallet

Returns:
    Transaction details
"""
wallet = self.get_or_create_wallet(user_id)

# Calculate expiry
now = datetime.now(self.IST)
expires_at = now + timedelta(days=expires_in_days)

# Record balance before
balance_before = wallet.total_balance

# Create wallet credit (line item)
credit = WalletCredit(
    user_id=user_id,
    amount=amount,
    remaining_balance=amount,
    source=source,
    source_id=source_id,
```

```
    earned_at=now,
    expires_at=expires_at
)
self.db.add(credit)
self.db.flush()


# Create transaction record
transaction_type = f"credit_earned_{source}"
transaction = WalletTransaction(
    user_id=user_id,
    wallet_id=wallet.id,
    transaction_type=transaction_type,
    amount=amount,
    source=source.split('_')[0],  # 'game', 'referral', etc.
    source_id=source_id,
    balance_before=balance_before,
    balance_after=balance_before + amount,
    expires_at=expires_at,
    description=f"Earned ₹{amount} from {source}"
)
self.db.add(transaction)
```

```
self.db.flush()

# Update credit with transaction reference
credit.transaction_id = transaction.id

# Update wallet balances
wallet.total_balance += amount
if source.startswith('game_'):
    wallet.game_credits += amount
elif source.startswith('referral'):
    wallet.referral_credits += amount
else:
    wallet.promo_credits += amount

wallet.lifetime_earned += amount
wallet.last_transaction_at = now
wallet.updated_at = now

self.db.commit()

return {
    "transaction_id": transaction.id,
```

```
            "credit_id": credit.id,
            "balance_before": balance_before,
            "balance_after": wallet.total_balance,
            "expires_at": expires_at
        }

    def use_credit(self, user_id: int, amount: int, booking_id: int) -> dict:
        """
        Use wallet credit for booking (FIFO - oldest credits first)

        Returns:
            Usage details
        """
        wallet = self.get_or_create_wallet(user_id)

        if wallet.total_balance < amount:
            raise ValueError(f"Insufficient balance. Available: ₹{wallet.total_balance}, Required:
₹{amount}")

        # Get active credits (FIFO - oldest first)
        active_credits = self.db.query(WalletCredit).filter(
            and_(
```

```
        WalletCredit.user_id == user_id,
        WalletCredit.expired == False,
        WalletCredit.fully_used == False,
        WalletCredit.remaining_balance > 0
    )
).order_by(WalletCredit.earned_at.asc()).all()

# Deduct from credits
remaining_to_deduct = amount
credits_used = []

for credit in active_credits:
    if remaining_to_deduct <= 0:
        break

    deduction = min(remaining_to_deduct, credit.remaining_balance)
    credit.remaining_balance -= deduction

    if credit.remaining_balance == 0:
        credit.fully_used = True
```

```
    credits_used.append(credit.id)
    remaining_to_deduct -= deduction

if remaining_to_deduct > 0:
    raise ValueError("Failed to deduct full amount")

# Record transaction
balance_before = wallet.total_balance
transaction = WalletTransaction(
    user_id=user_id,
    wallet_id=wallet.id,
    transaction_type="credit_used_booking",
    amount=-amount,  # Negative for deduction
    source="booking",
    source_id=booking_id,
    balance_before=balance_before,
    balance_after=balance_before - amount,
    description=f"Used ₹{amount} for session booking #{booking_id}"
)
self.db.add(transaction)

# Update wallet balance
```

```
    wallet.total_balance -= amount
    wallet.game_credits = max(0, wallet.game_credits - amount)
    wallet.lifetime_spent += amount
    wallet.last_transaction_at = datetime.now(self.IST)
    wallet.updated_at = datetime.now(self.IST)

    self.db.commit()

    return {
        "transaction_id": transaction.id,
        "credits_used": credits_used,
        "amount_deducted": amount,
        "balance_before": balance_before,
        "balance_after": wallet.total_balance
    }

def expire_old_credits(self) -> dict:
    """
    Expire credits older than 30 days (background job)

    Returns:
```

```
    Expiry stats
"""
now = datetime.now(self.IST)


# Find expired credits
expired_credits = self.db.query(WalletCredit).filter(
    and_(
        WalletCredit.expired == False,
        WalletCredit.expires_at <= now,
        WalletCredit.remaining_balance > 0
    )
).all()


total_expired_amount = 0
users_affected = set()


for credit in expired_credits:
    expired_amount = credit.remaining_balance


    # Mark as expired
    credit.expired = True
    credit.remaining_balance = 0
```

```
# Get wallet
wallet = self.db.query(UserWallet).filter(
    UserWallet.user_id == credit.user_id
).first()

if wallet:
    # Deduct from balance
    wallet.total_balance -= expired_amount
    wallet.game_credits -= expired_amount
    wallet.lifetime_expired += expired_amount

    # Record transaction
    transaction = WalletTransaction(
        user_id=credit.user_id,
        wallet_id=wallet.id,
        transaction_type="credit_expired",
        amount=-expired_amount,
        source=credit.source,
        source_id=credit.source_id,
        balance_before=wallet.total_balance + expired_amount,
```

```
            balance_after=wallet.total_balance,
            description=f"₹{expired_amount} credit expired (earned {credit.earned_at.date()})"
        )
        self.db.add(transaction)

        total_expired_amount += expired_amount
        users_affected.add(credit.user_id)

    self.db.commit()

    return {
        "total_expired": total_expired_amount,
        "credits_expired": len(expired_credits),
        "users_affected": len(users_affected)
    }

def get_expiring_soon(self, user_id: int, days: int = 7) -> List[dict]:
    """Get credits expiring within specified days"""
    now = datetime.now(self.IST)
    threshold = now + timedelta(days=days)

    expiring = self.db.query(WalletCredit).filter(
```

```
    and_(
        WalletCredit.user_id == user_id,
        WalletCredit.expired == False,
        WalletCredit.remaining_balance > 0,
        WalletCredit.expires_at <= threshold,
        WalletCredit.expires_at > now
    )
).order_by(WalletCredit.expires_at.asc()).all()


result = []
for credit in expiring:
    days_until = (credit.expires_at - now).days
    urgency = "high" if days_until <= 3 else "medium" if days_until <= 7 else "low"


    result.append({
        "credit_id": credit.id,
        "amount": credit.remaining_balance,
        "source": credit.source,
        "earned_on": credit.earned_at.date().isoformat(),
        "expires_at": credit.expires_at.isoformat(),
        "days_until_expiry": days_until,
```

```
            "urgency": urgency
        })

    return result
```

File: `api/endpoints/game.py`

```
from fastapi import APIRouter, Depends, HTTPException, Request
from sqlalchemy.orm import Session
from services.game_engine import GameEngine
from services.wallet_service import WalletService
from database import get_db
from auth import get_current_user
router = APIRouter(prefix="/api/v1/games/cricket", tags=["cricket-game"])
@router.get("/eligibility")
async def check_game_eligibility(
    current_user: dict = Depends(get_current_user),
    db: Session = Depends(get_db)
):
    """
    Check if user is eligible to play today's game

    Returns eligibility status and relevant data
```

```
"""
game_engine = GameEngine(db)
wallet_service = WalletService(db)

eligible, error, data = game_engine.check_eligibility(current_user['id'])

if not eligible:
    return {
        "eligible": False,
        "reason": error,
        **data
    }

# Get wallet balance
wallet = wallet_service.get_or_create_wallet(current_user['id'])
monthly_stats = game_engine._get_monthly_stats(current_user['id'])

return {
    "eligible": True,
    "game_status": data,
    "wallet": {
```

```
            "current_balance": wallet.total_balance,
            "this_month_winnings": monthly_stats['total_won'],
            "roi": monthly_stats['roi_percentage']
        }
    }
@router.post("/play")
async def play_cricket_game(
    request: Request,
    current_user: dict = Depends(get_current_user),
    db: Session = Depends(get_db)
):
    """
    Play today's cricket game

    Returns game outcome and wallet update
    """
    game_engine = GameEngine(db)

    # Get client info
    ip_address = request.client.host
    user_agent = request.headers.get("user-agent")
```

```
    result = game_engine.play_game(
        user_id=current_user['id'],
        ip_address=ip_address,
        user_agent=user_agent
    )

    if not result['success']:
        raise HTTPException(
            status_code=400,
            detail={
                "error": result['error'],
                "message": result['data'].get('message', 'Cannot play game')
            }
        )

    return result
@router.get("/winners")
async def get_recent_winners(
    limit: int = 20,
    db: Session = Depends(get_db)
):
```

```
"""
Get recent game winners (for social proof feed)

Privacy: Shows only first name + last initial
"""
from models.database import DailyGamePlay, User
from datetime import datetime, timedelta
import pytz

IST = pytz.timezone('Asia/Kolkata')
today = datetime.now(IST).date()

winners = db.query(DailyGamePlay, User).join(
    User, DailyGamePlay.user_id == User.id
).filter(
    DailyGamePlay.game_date == today,
    DailyGamePlay.outcome.in_(['sixer', 'four'])
).order_by(
    DailyGamePlay.played_at.desc()
).limit(limit).all()

result = []
```

```
    for play, user in winners:

        # Privacy: first name + last initial

        display_name = f"{user.first_name} {user.last_name[0]}." if user.last_name else
user.first_name



        result.append({

            "display_name": display_name,

            "outcome": play.outcome,

            "amount_won": play.credit_amount,

            "time_ago": _format_time_ago(play.played_at),

            "emoji": "🚀" if play.outcome == "sixer" else "🎯"

        })



    return {

        "winners": result,

        "total_today": len(winners)

    }

def _format_time_ago(dt: datetime) -> str:

    """Format datetime as 'X mins ago'"""

    import pytz

    IST = pytz.timezone('Asia/Kolkata')

    now = datetime.now(IST)
```

```
    if dt.tzinfo is None:
        dt = IST.localize(dt)

    diff = now - dt

    if diff.seconds < 60:
        return "just now"
    elif diff.seconds < 3600:
        mins = diff.seconds // 60
        return f"{mins} min{'s' if mins > 1 else ''} ago"
    elif diff.seconds < 86400:
        hours = diff.seconds // 3600
        return f"{hours} hour{'s' if hours > 1 else ''} ago"
    else:
        return dt.strftime("%b %d")
```

FRONTEND CODE (REACT)

File: `components/CricketGame.jsx`

```
import React, { useState, useEffect } from 'react';

import { motion, AnimatePresence } from 'framer-motion';
```

```
import { api } from '../services/api';

import './CricketGame.css';

const CricketGame = () => {

  const [eligibility, setEligibility] = useState(null);

  const [loading, setLoading] = useState(true);

  const [playing, setPlaying] = useState(false);

  const [result, setResult] = useState(null);

  const [timeRemaining, setTimeRemaining] = useState('');

  useEffect(() => {

    checkEligibility();

    const interval = setInterval(updateTimeRemaining, 1000);

    return () => clearInterval(interval);

  }, []);

  const checkEligibility = async () => {

    try {

      const response = await api.get('/games/cricket/eligibility');

      setEligibility(response.data);

      setLoading(false);

    } catch (error) {

      console.error('Failed to check eligibility:', error);

      setLoading(false);
```

```
  }
};
const updateTimeRemaining = () => {
  if (!eligibility?.game_status?.time_remaining_seconds) return;

  const seconds = eligibility.game_status.time_remaining_seconds;
  const hours = Math.floor(seconds / 3600);
  const minutes = Math.floor((seconds % 3600) / 60);

  setTimeRemaining(`${hours}h ${minutes}m left`);
};
const playGame = async () => {
  setPlaying(true);

  try {
    // Simulate ball animation delay
    await new Promise(resolve => setTimeout(resolve, 2000));

    const response = await api.post('/games/cricket/play');
    setResult(response.data);
  } catch (error) {
    console.error('Failed to play game:', error);
```

```
    alert(error.response?.data?.detail?.message || 'Failed to play game');
  } finally {
    setPlaying(false);
  }
};
if (loading) {
  return <div className="game-loading">Loading game...</div>;
}
// Not eligible - show upgrade prompt
if (!eligibility?.eligible) {
  return <SubscribePrompt reason={eligibility?.reason} />;
}
// Show result if just played
if (result) {
  return <GameResult result={result} onClose={() => setResult(null)} />;
}
// Game UI
return (
  <div className="cricket-game">
    <div className="game-header">
      <h2>🏏 HIT A SIXER 🏏</h2>
```

```
  <p className="tagline">Play before 6 PM today!</p>
  <div className="time-remaining">⏰ {timeRemaining}</div>
</div>
<div className="wallet-display">
  <div className="wallet-label">YOUR WALLET</div>
  <div className="wallet-amount">₹{eligibility.wallet.current_balance}</div>
  <div className="wallet-roi">
    This month: ₹{eligibility.wallet.this_month_winnings}
    ({eligibility.wallet.roi}% ROI)
  </div>
</div>
<div className="game-arena">
  <AnimatePresence>
    {playing ? (
      <motion.div
        className="ball-animation"
        initial={{ x: -100, y: 0 }}
        animate={{ x: 300, y: -50 }}
        transition={{ duration: 2 }}
      >
        🔴
      </motion.div>
```

```
    ) : (
      <motion.div
        className="bat-icon"
        whileHover={{ scale: 1.1 }}
        whileTap={{ scale: 0.95 }}
      >
        🏏
      </motion.div>
    )}
  </AnimatePresence>
</div>
<button
  className="play-button"
  onClick={playGame}
  disabled={playing}
>
  {playing ? 'SWINGING...' : 'SWING! 🎯'}
</button>
<div className="prizes-list">
  <div className="prize-item">
    <span className="prize-emoji">🚀</span>
```

```
          <span className="prize-name">SIXER</span>
          <span className="prize-value">₹100</span>
          <span className="prize-chance">(4%)</span>
        </div>
        <div className="prize-item">
          <span className="prize-emoji">🎯</span>
          <span className="prize-name">FOUR</span>
          <span className="prize-value">₹50</span>
          <span className="prize-chance">(8%)</span>
        </div>
        <div className="prize-item">
          <span className="prize-emoji">💰</span>
          <span className="prize-name">OUT</span>
          <span className="prize-value">₹10</span>
          <span className="prize-chance">(88%)</span>
        </div>
      </div>
      <WinnersFeed />
    </div>
  );
};
const GameResult = ({ result, onClose }) => {
```

```
const outcomeConfig = {
  sixer: { emoji: '🚀', color: '#00ff88', title: 'SIXER!!!' },
  four: { emoji: '🎯', color: '#ffaa00', title: 'FOUR!' },
  out: { emoji: '💰', color: '#0088ff', title: 'OUT' }
};
const config = outcomeConfig[result.outcome];
return (
  <motion.div
    className="game-result"
    initial={{ scale: 0 }}
    animate={{ scale: 1 }}
    transition={{ type: 'spring', duration: 0.5 }}
  >
    <div className="result-content" style={{ borderColor: config.color }}>
      <div className="result-emoji">{config.emoji}</div>
      <h2 className="result-title">{config.title}</h2>


      <div className="result-prize">
        <div className="prize-amount">₹{result.prize.amount}</div>
        <div className="prize-description">{result.prize.description}</div>
      </div>
```

```
<div className="wallet-update">
  <div className="wallet-row">
    <span>Before:</span>
    <span>₹{result.wallet.previous_balance}</span>
  </div>
  <div className="wallet-row highlight">
    <span>Won:</span>
    <span>+₹{result.wallet.credit_added} {config.emoji}</span>
  </div>
  <div className="wallet-row total">
    <span>Now:</span>
    <span>₹{result.wallet.new_balance}</span>
  </div>
</div>
<div className="expiry-notice">
  Expires in 30 days - use it or lose it!
</div>
<div className="result-actions">
  <button className="btn-primary" onClick={() => window.location.href = '/book-session'}>
    📅 BOOK SESSION
  </button>
  <button className="btn-secondary" onClick={() => window.location.href = '/wallet'}>
```

```
          💳 VIEW WALLET
        </button>
        <button className="btn-tertiary" onClick={onClose}>
          Close
        </button>
      </div>
      <div className="monthly-stats">
        <div>Games this month: {result.monthly_stats.games_played}</div>
        <div>Total won: ₹{result.monthly_stats.total_won}</div>
        <div>ROI: {result.monthly_stats.roi_percentage}% 🚀</div>
      </div>
    </div>
  </motion.div>
);
};
const WinnersFeed = () => {
  const [winners, setWinners] = useState([]);
  useEffect(() => {
    fetchWinners();
    const interval = setInterval(fetchWinners, 30000); // Refresh every 30s
    return () => clearInterval(interval);
```

```
}, []);

const fetchWinners = async () => {

  try {

    const response = await api.get('/games/cricket/winners?limit=10');

    setWinners(response.data.winners);

  } catch (error) {

    console.error('Failed to fetch winners:', error);

  }

};

return (

  <div className="winners-feed">

    <h3>🏆 TODAY'S WINNERS 🏆</h3>

    <div className="winners-list">

      {winners.map((winner, index) => (

        <div key={index} className="winner-item">

          <span className="winner-emoji">{winner.emoji}</span>

          <span className="winner-name">{winner.display_name}</span>

          <span className="winner-outcome">{winner.outcome.toUpperCase()}</span>

          <span className="winner-time">{winner.time_ago}</span>

        </div>

      ))}

    </div>
```

```
    </div>
  );
};
const SubscribePrompt = ({ reason }) => {
  return (
    <div className="subscribe-prompt">
      <div className="lock-icon">🔒 </div>
      <h2>🏏 HIT A SIXER 🏏</h2>
      <p className="subtitle">Subscriber Exclusive Feature</p>


      <div className="benefits">
        <div className="benefit-item">
          <span className="check">✓ </span>
          <span>Win up to ₹100 daily</span>
        </div>
        <div className="benefit-item">
          <span className="check">✓ </span>
          <span>Average ₹450/month winnings</span>
        </div>
        <div className="benefit-item">
          <span className="check">✓ </span>
```

```
        <span>354% ROI guaranteed</span>
      </div>
      <div className="benefit-item">
        <span className="check">✓ </span>
        <span>Reduce therapy costs by 45%</span>
      </div>
    </div>
    <div className="pricing">
      <div className="price">₹99/month</div>
      <div className="price-description">Break-even in just 10 days!</div>
    </div>
    <button className="subscribe-button" onClick={() => window.location.href = '/subscribe'}>
      SUBSCRIBE & START WINNING 🚀
    </button>
    <div className="guarantee">30-day money-back guarantee</div>
  </div>
 );
};
export default CricketGame;
```

BACKGROUND JOBS

File: `jobs/expire_credits.py`

```
from celery import Celery

from datetime import datetime

import pytz

from database import SessionLocal

from services.wallet_service import WalletService

from services.notification_service import NotificationService

celery = Celery('mans360', broker='redis://localhost:6379/0')

@celery.task

def expire_old_credits():

    """

    Daily job to expire credits older than 30 days

    Runs at midnight IST

    """

    db = SessionLocal()

    try:

        wallet_service = WalletService(db)

        notification_service = NotificationService(db)



        # Expire credits
```

```
        result = wallet_service.expire_old_credits()

        print(f"Credits expired: {result['credits_expired']}")
        print(f"Amount expired: ₹{result['total_expired']}")
        print(f"Users affected: {result['users_affected']}")

        # Notify users who lost credits (if > ₹50)
        # ... notification logic ...

        return result
    finally:
        db.close()
@celery.task
def send_expiry_warnings():
    """
    Send warnings 7 days before credit expiry
    Runs daily at 10 AM IST
    """
    db = SessionLocal()
    try:
        # Get all users with credits expiring in 7 days
```

```
        # Send push notification / email
        pass
    finally:
        db.close()
@celery.task
def send_morning_game_reminder():
    """
    Send morning reminder to play game
    Runs daily at 10 AM IST
    """
    db = SessionLocal()
    try:
        # Get all subscribers who haven't played today
        # Send push notification
        pass
    finally:
        db.close()
# Schedule tasks
celery.conf.beat_schedule = {
    'expire-credits-midnight': {
        'task': 'jobs.expire_credits.expire_old_credits',
        'schedule': crontab(hour=0, minute=0),  # Midnight IST
```

```
    },
    'expiry-warnings-morning': {
        'task': 'jobs.expire_credits.send_expiry_warnings',
        'schedule': crontab(hour=10, minute=0),  # 10 AM IST
    },
    'game-reminder-morning': {
        'task': 'jobs.expire_credits.send_morning_game_reminder',
        'schedule': crontab(hour=10, minute=0),  # 10 AM IST
    },
}
```

TESTING SPECIFICATIONS

Unit Tests

```
# tests/test_game_engine.py
import pytest
from services.game_engine import GameEngine
def test_probability_distribution():
    """Test that probabilities are within expected ranges"""
    engine = GameEngine(db)


    outcomes = {'sixer': 0, 'four': 0, 'out': 0}
```

```
    iterations = 10000

    for _ in range(iterations):
        outcome, _ = engine.generate_outcome()
        outcomes[outcome.value] += 1

    # Check percentages (with 1% tolerance)
    sixer_pct = (outcomes['sixer'] / iterations) * 100
    four_pct = (outcomes['four'] / iterations) * 100
    out_pct = (outcomes['out'] / iterations) * 100

    assert 3.0 <= sixer_pct <= 5.0, f"SIXER: {sixer_pct}% (expected 4%)"
    assert 7.0 <= four_pct <= 9.0, f"FOUR: {four_pct}% (expected 8%)"
    assert 87.0 <= out_pct <= 89.0, f"OUT: {out_pct}% (expected 88%)"
def test_time_window_enforcement():
    """Test that game closes after 6 PM"""
    from datetime import datetime
    import pytz

    engine = GameEngine(db)
    IST = pytz.timezone('Asia/Kolkata')
```

```
    # Mock time to 6:30 PM
    with freeze_time(IST.localize(datetime(2026, 1, 17, 18, 30))):
        eligible, error, _ = engine.check_eligibility(user_id=1)
        assert not eligible
        assert error == "GAME_CLOSED"
def test_daily_limit_enforcement():
    """Test that user can only play once per day"""
    engine = GameEngine(db)

    # First play - should succeed
    result1 = engine.play_game(user_id=1)
    assert result1['success'] == True

    # Second play same day - should fail
    result2 = engine.play_game(user_id=1)
    assert result2['success'] == False
    assert result2['error'] == "ALREADY_PLAYED_TODAY"
def test_subscription_requirement():
    """Test that non-subscribers cannot play"""
    engine = GameEngine(db)
```

```
    # User without active subscription
    eligible, error, _ = engine.check_eligibility(user_id=999)
    assert not eligible
    assert error == "SUBSCRIPTION_REQUIRED"
def test_wallet_credit_addition():
    """Test that credits are correctly added to wallet"""
    from services.wallet_service import WalletService

    wallet_service = WalletService(db)

    # Add credit
    result = wallet_service.add_credit(
        user_id=1,
        amount=100,
        source="game_sixer",
        source_id=1,
        expires_in_days=30
    )

    # Verify wallet balance
    wallet = wallet_service.get_or_create_wallet(user_id=1)
```

```
    assert wallet.total_balance == result['balance_after']

    assert result['balance_after'] - result['balance_before'] == 100

def test_credit_expiry():

    """Test that credits expire after 30 days"""

    from datetime import timedelta


    wallet_service = WalletService(db)


    # Add credit that expires today

    with freeze_time(datetime.now() - timedelta(days=31)):

        wallet_service.add_credit(

            user_id=1,

            amount=50,

            source="game_four",

            source_id=1,

            expires_in_days=30

        )


    # Run expiry job

    result = wallet_service.expire_old_credits()


    assert result['total_expired'] == 50
```

```
assert result['credits_expired'] == 1
```

TASK BREAKDOWN

Backend Tasks (8 points)

Task 1.1: Database Schema Setup (1 point)

- [ ] Create all 6 tables
- [ ] Add indexes
- [ ] Set up foreign keys
- [ ] Write migration scripts
- [ ] Verify schema in dev environment

Task 1.2: Game Engine Service (2 points)

- [ ] Implement eligibility checking
- [ ] Build cryptographically secure probability engine
- [ ] Add IST timezone handling
- [ ] Create outcome generation logic
- [ ] Write unit tests (>90% coverage)

Task 1.3: Wallet Service (2 points)

- [ ] Implement credit addition (FIFO)
- [ ] Build credit usage logic
- [ ] Add expiry handling
- [ ] Create transaction logging
- [ ] Write integration tests

Task 1.4: API Endpoints (2 points)

- [ ] GET /eligibility endpoint
- [ ] POST /play endpoint
- [ ] GET /winners endpoint
- [ ] Error handling
- [ ] Request validation

- [ ] API documentation (OpenAPI)

Task 1.5: Background Jobs (1 point)

- [ ] Credit expiry cron (midnight IST)
- [ ] Expiry warning job (7 days before)
- [ ] Morning reminder job (10 AM IST)
- [ ] Job monitoring & logging

Frontend Tasks (5 points)

Task 2.1: Game Component UI (2 points)

- [ ] Cricket game interface
- [ ] Ball swing animation
- [ ] Play button interaction
- [ ] Wallet display widget
- [ ] Mobile responsive design

Task 2.2: Result Screens (1 point)

- [ ] SIXER celebration screen
- [ ] FOUR result screen
- [ ] OUT result screen
- [ ] Wallet balance update animation
- [ ] Share buttons integration

Task 2.3: Subscribe Prompt (1 point)

- [ ] Non-subscriber gate screen
- [ ] Upgrade benefits list
- [ ] Pricing display
- [ ] CTA button integration
- [ ] A/B testing hooks

Task 2.4: Winners Feed (1 point)

- [ ] Real-time winners list
- [ ] Privacy-safe name display

- [ ] Auto-refresh (30s interval)
- [ ] Animation on new winner

DEFINITION OF DONE

Technical Requirements

- [ ] All database tables created and migrated
- [ ] All API endpoints functional and documented
- [ ] Unit test coverage >90%
- [ ] Integration tests passing
- [ ] Load testing completed (1,000 concurrent plays)
- [ ] Security audit passed
- [ ] Code review approved

Functional Requirements

- [ ] Subscribers can play once per day
- [ ] Non-subscribers see upgrade prompt
- [ ] Game closes at 6 PM IST
- [ ] Probabilities match spec (4%, 8%, 88%)
- [ ] Credits added to wallet instantly
- [ ] Credits expire after 30 days
- [ ] Wallet auto-applies at checkout

Quality Requirements

- [ ] No P0/P1 bugs
- [ ] Response time <200ms for play endpoint
- [ ] Database queries optimized
- [ ] Error handling comprehensive
- [ ] Logging implemented
- [ ] Monitoring dashboards created

Documentation

- [ ] API documentation complete
- [ ] Database schema documented
- [ ] README updated

- [ ] Deployment guide written
- [ ] Runbook for on-call created

DEPLOYMENT CHECKLIST

Pre-Deployment

- [ ] Run all tests in staging
- [ ] Verify database migrations
- [ ] Test background jobs
- [ ] Load test with 1,000 concurrent users
- [ ] Security scan
- [ ] Backup database

Deployment

- [ ] Apply database migrations
- [ ] Deploy backend code
- [ ] Deploy frontend code
- [ ] Start background job workers
- [ ] Verify cron schedules (IST timezone)
- [ ] Enable feature flag for 10% users

Post-Deployment

- [ ] Monitor error rates
- [ ] Check probability distribution
- [ ] Verify wallet balances
- [ ] Test full user journey
- [ ] Monitor performance metrics
- [ ] Gradual rollout to 100%

MONITORING & ALERTS

Key Metrics to Monitor

Game Metrics:

- Plays per day
- Probability distribution (4%, 8%, 88%)
- Play rate (% of subscribers)
- Time-of-day distribution

Financial Metrics:

- Credits issued per day
- Credits redeemed per day
- Breakage rate (% expired)
- Cost per play
- Revenue impact

Technical Metrics:

- API response time (<200ms)
- Error rate (<0.1%)
- Database query time (<50ms)
- Background job success rate (>99%)

Alerts to Configure

Critical:

- Game down (can't play)
- Database connection failed
- Probability skewed >2% from target
- Credits not being added to wallet

Warning:

- Play rate drops >20%
- Response time >500ms
- Background job failures
- Unusual spike in plays (bot detection)

RISKS & MITIGATION

Risk 1: Probability Manipulation

Risk: Users try to manipulate outcomes **Mitigation:**

- Use cryptographically secure randomization
- Server-side only (no client-side RNG)
- Audit trail in database
- Device fingerprinting
- Rate limiting

Risk 2: Multi-Account Abuse

Risk: Users create multiple accounts **Mitigation:**

- Subscription required (payment verification)
- Device fingerprinting
- IP tracking
- Phone number verification
- Manual review of suspicious patterns

Risk 3: Timezone Issues

Risk: Wrong timezone causes incorrect closing time **Mitigation:**

- All times in IST (pytz enforced)
- Extensive timezone testing
- Server clock monitoring
- User sees local time conversion

Risk 4: High Redemption Cost

Risk: More users redeem than expected **Mitigation:**

- Conservative estimates (85% redemption)
- Real-time cost monitoring
- Alert if cost >120% of budget
- Can reduce OUT from ₹10 to ₹5 if needed

APPENDIX

Sample Database Queries

Get daily statistics:

```
SELECT

    game_date,

    COUNT(*) as total_plays,

    SUM(CASE WHEN outcome = 'sixer' THEN 1 ELSE 0 END) as sixers,

    SUM(CASE WHEN outcome = 'four' THEN 1 ELSE 0 END) as fours,

    SUM(CASE WHEN outcome = 'out' THEN 1 ELSE 0 END) as outs,

    SUM(credit_amount) as total_credits_issued

FROM daily_game_plays

WHERE game_date >= CURRENT_DATE - INTERVAL '7 days'

GROUP BY game_date

ORDER BY game_date DESC;
```

Get user's wallet balance:

```
SELECT

    u.id,

    u.first_name,

    uw.total_balance,

    uw.game_credits,

    uw.lifetime_earned,
```

```
    uw.lifetime_spent
FROM users u
JOIN user_wallet uw ON u.id = uw.user_id
WHERE u.id = :user_id;
```

Find expiring credits:

```
SELECT
    wc.id,
    wc.user_id,
    wc.amount,
    wc.remaining_balance,
    wc.expires_at,
    DATE_PART('day', wc.expires_at - NOW()) as days_until_expiry
FROM wallet_credits wc
WHERE wc.expired = FALSE
  AND wc.fully_used = FALSE
  AND wc.expires_at <= NOW() + INTERVAL '7 days'
ORDER BY wc.expires_at ASC;
```

END OF STORY 11.5 SPECIFICATION